

RSA_APB — 32-bit RSA Coprocessor with APB Interface

1. Overview

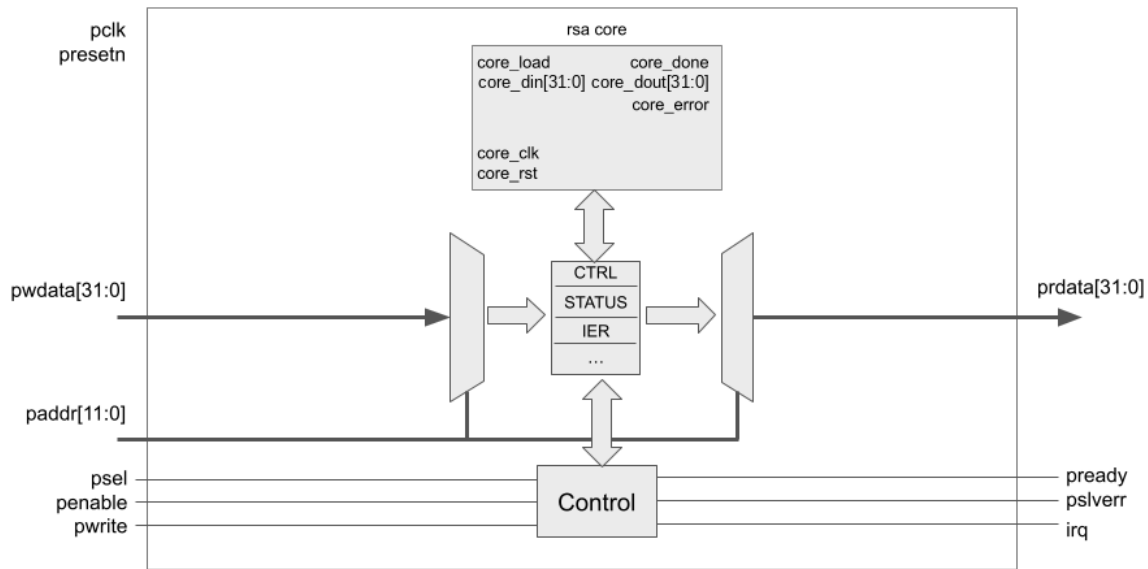
RSA_APB computes $C = M^e \bmod n$ on 32-bit operands and is controlled by software through an AMBA 3 APB (APB3) slave port. It wraps the existing `rsa_core` block: software writes the message, exponent and modulus to registers, sets `START`, and reads the result or an error flag when the core finishes.

The wrapper adds data, control and status registers that feed the three operands to `rsa_core` over its 32-bit `core_din` port, result capture, sticky status flags and an interrupt output.

Features

- Modular exponentiation $C = M^e \bmod n$ with 32-bit message, exponent and modulus
- AMBA 3 APB slave: 32-bit data bus, 12-bit address (4 KB window), zero wait states, `PSLVERR` error response
- Operand registers written in any order; a single `START` bit launches the operation
- Internal control block generates the three `core_load` cycles (M, then e, then n) required by `rsa_core`
- Result is registered on `core_done` and held until the next operation
- Sticky `DONE` and `ERROR` flags (write-1-to-clear) and divide-by-zero detection when $n = 0$
- Level-sensitive, active-high interrupt with individual enables for `DONE` and `ERROR`
- Software reset of the core and internal control logic without resetting the APB bus
- Single clock domain: `rsa_core` runs on `PCLK`

2. Block Diagram



RSA_APB block diagram · APB wrapper around rsa_core

3. Signal Description

The external interface of RSA_APB is the APB3 slave port plus one interrupt line. The `rsa_core` pins become internal nets driven and observed by the wrapper.

3.1 APB3 slave interface

Signal	Width	Direction	Active	Description
PCLK	1	input	rising edge	APB clock. Also clocks <code>rsa_core</code> (<code>core_clk</code>).
PRESETn	1	input	LOW	APB reset. Resets all registers, the sequencer and <code>rsa_core</code> .
PSEL	1	input	HIGH	Slave select from the APB decoder.
PENABLE	1	input	HIGH	Marks the second (access) phase of a transfer.
PWRITE	1	input	HIGH	1 = write transfer, 0 = read transfer.
PADDR	12	input	-	Byte address. Bits [1:0] are ignored; accesses are word aligned.
PWDATA	32	input	-	Write data.
PRDATA	32	output	-	Read data. Reserved bits read as 0.
PREADY	1	output	HIGH	Tied HIGH: every transfer completes with zero wait states.
PSLVERR	1	output	HIGH	Transfer error, valid in the access phase (see section 6.7).
irq	1	output	HIGH	Interrupt request, level sensitive (see IER register).

3.2 Internal rsa_core interface

These signals come from the reference design. They are not package pins of RSA_APB.

Signal	Width	Direction (at rsa_core)	Active	Description
core_clk	1	input	rising edge	System clock, connected to PCLK.
core_rst	1	input	HIGH	Synchronous reset. Driven by the reset synchroniser from PRESETn OR CTRL.SRST.
core_load	1	input	HIGH	When HIGH, core_din is registered. Three operands must be loaded to start an operation.
core_din	32	input	-	Operand data. Loaded in order: message M, exponent e, modulus n.
core_done	1	output	HIGH	Operation complete and result valid on core_dout. HIGH for one core_clk cycle.
core_error	1	output	HIGH	Error (division by zero, n = 0). HIGH for one core_clk cycle.
core_dout	32	output	-	Result $C = M^e \bmod n$.

4. Functional Description

An operation runs in four stages: operands are loaded via APB interface, START launches the internal control logic that feeds rsa_core, and the result or error is captured into STATUS and RESULT.

4.1 RSA operation

rsa_core computes the modular exponentiation below on 32-bit unsigned values.

$$C = M^e \bmod n, \quad 0 \leq M, e, n \leq 2^{32} - 1$$

For a meaningful RSA result software should keep $M < n$. A modulus of $n = 0$ is a division by zero: the core asserts core_error instead of core_done, and RESULT is not updated. Computation latency (T_core, in clock cycles from the third load to core_done) is set by the rsa_core implementation.

4.2 Operand registers

MSG, EXP and MOD each hold one 32-bit operand. They can be written in any order and keep their value after an operation, so a new operation that changes only one operand needs only one register write. Writes to these registers while STATUS.BUSY = 1 are rejected with PSLVERR and do not change the register.

4.3 Load control logic

Writing 1 to CTRL.START while the block is idle starts the internal control logic. It drives core_load HIGH for one cycle with MSG on core_din, then with EXP, then with MOD, inserting LOAD_GAP idle cycles (default 0) between loads; rsa_core accepts back-to-back loads, so no gap is needed. It then waits for core_done or core_error. STATUS.BUSY is 1 from the START write until the cycle after core_done or core_error.

4.4 Completion and error handling

- On `core_done`, `core_dout` is captured into `RESULT` and `STATUS.DONE` is set.
- On `core_error`, `STATUS.ERROR` is set and `RESULT` keeps its previous value.
- `DONE` and `ERROR` are sticky until software writes 1 to the bit. Starting a new operation also clears both.
- `irq = (STATUS.DONE AND IER.DONE_IE) OR (STATUS.ERROR AND IER.ERR_IE)`.

4.5 Reset

`PRESETn` LOW resets every register to its reset value, returns the sequencer to IDLE and asserts `core_rst`. Writing 1 to `CTRL.SRST` asserts `core_rst` for one cycle and returns the sequencer to IDLE, aborting any operation in progress. `SRST` leaves `MSG`, `EXP`, `MOD`, `RESULT` and `IER` unchanged and clears `STATUS`. Because `core_rst` is synchronous, `PRESETn` is deasserted through a two-flop synchroniser before reaching it.

5. Register Map

`RSA_APB` has eight 32-bit registers at offsets 0x00 to 0x1C from the base address assigned by the system integrator. Offsets 0x20 to 0xFFC are reserved.

Offset	Name	Access	Reset	Description
0x00	CTRL	WO	0x0000_0000	Start and software reset (self-clearing bits)
0x04	STATUS	RO / W1C	0x0000_0000	Busy, done and error flags
0x08	IER	RW	0x0000_0000	Interrupt enables
0x0C	MSG	RW	0x0000_0000	Message <code>M[31:0]</code>
0x10	EXP	RW	0x0000_0000	Exponent <code>e[31:0]</code>
0x14	MOD	RW	0x0000_0000	Modulus <code>n[31:0]</code>
0x18	RESULT	RO	0x0000_0000	Ciphertext <code>C[31:0] = M^e mod n</code>
0x1C	ID	RO	0x5253_0200	Block identifier and version

Access codes: RW = read/write, RO = read only, WO = write only (reads return 0), W1C = write 1 to clear.

6. Register Descriptions

All bits not listed are reserved: they read as 0 and must be written as 0.

6.1 CTRL (0x00)

Bits	Field	Access	Reset	Description
31:2	-	-	0	Reserved
1	SRST	WO	0	Write 1: one-cycle <code>core_rst</code> , sequencer to IDLE, <code>STATUS</code> cleared. Self-clearing.
0	START	WO	0	Write 1: clear <code>DONE</code> and <code>ERROR</code> , then load <code>MSG</code> , <code>EXP</code> , <code>MOD</code> into <code>rsa_core</code> . Self-clearing. Ignored with <code>PSLVERR</code> if <code>BUSY</code> = 1.

If `SRST` and `START` are written as 1 in the same transfer, `SRST` takes effect and `START` is ignored.

6.2 STATUS (0x04)

Bits	Field	Access	Reset	Description
31:3	-	-	0	Reserved
2	ERROR	W1C	0	Set when core_error pulses (n = 0). Write 1 to clear.
1	DONE	W1C	0	Set when core_done pulses and RESULT is updated. Write 1 to clear.
0	BUSY	RO	0	1 from the START write until DONE or ERROR is set.

6.3 IER (0x08)

Bits	Field	Access	Reset	Description
31:2	-	-	0	Reserved
1	ERR_IE	RW	0	1 = STATUS.ERROR drives irq
0	DONE_IE	RW	0	1 = STATUS.DONE drives irq

6.4 MSG (0x0C), EXP (0x10), MOD (0x14)

Bits	Field	Access	Reset	Description
31:0	VALUE	RW	0	Operand: M in MSG, e in EXP, n in MOD. Write rejected with PSLVERR while BUSY = 1.

6.5 RESULT (0x18)

Bits	Field	Access	Reset	Description
31:0	C	RO	0	core_dout captured on the last core_done. Unchanged by an error.

6.6 ID (0x1C)

Bits	Field	Access	Reset	Description
31:16	BLOCK	RO	0x5253	ASCII "RS"
15:8	MAJOR	RO	0x02	Major version
7:0	MINOR	RO	0x00	Minor version

6.7 PSLVERR conditions

PSLVERR is asserted in the access phase, and the transfer has no effect, when any of these is true:

- A write targets RESULT or ID
- A write to CTRL with START = 1, or to MSG, EXP or MOD, occurs while BUSY = 1

Reads of CTRL return 0 and never raise PSLVERR.

7. Timing: APB transfers

Every APB transfer takes two PCLK cycles (setup + access) because PREADY is always HIGH. An operation then takes $4 + T_{\text{core}}$ cycles with the default LOAD_GAP of 0.

7.1 APB Write

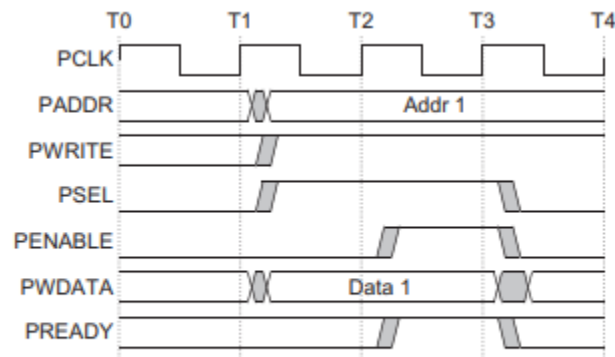


Figure 2-1 Write transfer with no wait states

The write transfer starts with the address, write data, write signal and select signal all changing after the rising edge of the clock. The first clock cycle of the transfer is called the Setup phase. After the following clock edge the enable signal is asserted, PENABLE, and this indicates that the Access phase is taking place. The address, data and control signals all remain valid throughout the Access phase. The transfer completes at the end of this cycle.

The enable signal, PENABLE, is deasserted at the end of the transfer. The select signal, PSELx, also goes LOW unless the transfer is to be followed immediately by another transfer to the same peripheral.

7.2 APB Read

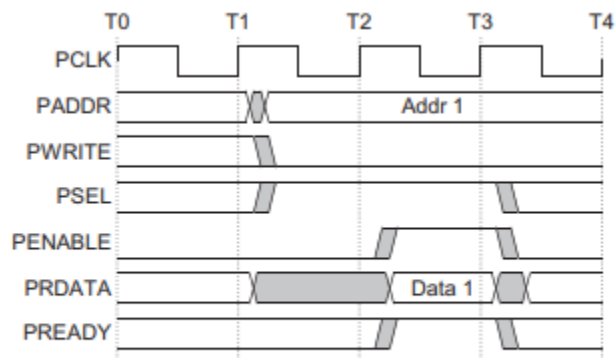


Figure 2-3 Read transfer with no wait states